Универзитет у Новом Саду
Факултет техничких наука
Департман за рачунарство и аутоматику

Примена Minimax и Alpha-beta pruning алгоритама над игром 4 Connect

Предмет: Операциона истраживања

Марија Тодоровић IN 13/2022
Ненад Лукић IN 17/2022
Студијски програм: Информациони инжењеринг
Нови Сад, фебруар 2026.

Садржај:

Увод **2**
Архитектура система **2**
Графички кориснички интерфејс за игру **3**
Опис Minimax алгоритма **4**
Опис Alpha-beta pruning алгоритма **5**
Опис коришћене хеуристике **6**
Стратешка контрола центра 6
Евалуација прозора 6
Поређење времена извршавања **7**
Minimax алгоритам 7
Alpha-beta pruning алгоритам 8
Поређење алгоритама 10
Закључак **11**

Увод

4 Connect је класична стратешка игра за два играча која се традиционално игра на вертикално постављеној табли. У овој имплементацији, табла је димензија 6x6. Ток игре је такав да играчи наизменично убацују дискове у једну од 6 колона, а диск заузима најниже место у одабраној колони.
Основни циљ сваког играча јесте да пре свог противника формира непрекидан низ од четири своја диска. Овај низ се може формирати на три начина: хоризонтално, вертикално и дијагонално. Партија се завршава победом једног играча који је везао 4 диска у низу или нерешено у случају да се цела табла попунила без победника.
Ова игра спада у категорију игара са савршеним информацијама, из разлога што оба играча у сваком тренутку виде стање на табли.
Овим пројектом смо се потрудили да решимо проблем претраге простора стања, јер сваки нови потез генерише нове гране у стаблу игре, а број могућих комбинација експоненцијално расте. У овом пројекту ћемо употребом алгоритама који симулирају будуће исходе и употребом хеуристила креирати агента који ће доносити рационалне, али и оптималне одлуке.

Архитектура система

Архитектура система је заснована на модуларном приступу, где су логика игре, графички интерфејс као и логика која стоји иза агената јасно раздвојени у сопственим функцијама. Систем је имплементиран употребом *python* верзије 3.12, због своје подршке за многе библиотеке.
Систем се ослања највише на употребу наредне две кључне библиотеке:

- *NumPy* библиотека која се користи за интерну репрезентацију табле управо због своје подршке за ефикасан рад са матрицама употребом: брзе индексације, методе *copy()* и *“slicing”* оператора.

- *Pygame* библиотека која је задужена за мултимедијални део у шта спадају исцртавање графичких елемената система и обрада корисничких уноса.

Можемо издвојити четири функционална блока унутар саме архитектуре:

- Репрезентација стања, где је дефинисана матрица димензија 6x6 чија поља могу имати неку од следећих вредности које означавају ком играчу диск припада: 0 - празно, 1 - играч или 2 - агент.
- Логика игре, где су дефинисане функције као што су: *drop_piece* - служи како би се одиграо потез, *is_valid_location* - провера да ли је одабрани потез валидан и *winning_move* - одређује да ли негде на табли постоје 4 у низу.
- Блок агента, који се ослања на употребу *Minimax* и *Alpha-beta pruning* алгоритама како би агент донео одлуку о наредном потезу.
- Кориснички интефејс, који садржи мени систем где играч може одабрати алгоритам и његову тежину, против којег ће играти у наредној партији, али и боју дискова. Овде такође спада и графичка репрезентација табле на којој се игра партија.

Када играч кликне на одређену колону, податак о локацији колоне се шаље логици игре која проверава његову валидност и ажурира таблу, након чега је ред на агенту да одигра потез, који ће одабрати у односу на дефинисану хеуристику. Након сваког потеза, екран ће се ажурирати и пружити информације о тренутном стању табле.

Графички кориснички интерфејс за игру

Интерфејс је дизајниран са намером да кориснику пружи интуитивно искуство, али такође и да му представи релевантне статистичке податке о раду самог агента. Постоје две врсте дискова: црвени и плави.

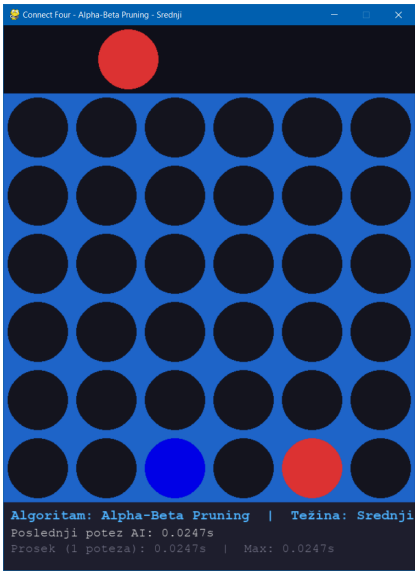
Пре почетка саме игре, кориснику се приказује почетни мени, на ком он може да одабере: против ког од два имплементирана алгоритма ће играти и може одабрати тежину агента против којег игра, као и боју диска, где играч са црвеним дисковима игра први.

Слика 1: Главни мени апликације

Игра се одвија на матрици димензија 6x6, где сваки правоугаоник има отвор за дискове, уз “пливајући” диск који прати покрете миша и на основу клика на одређеној локацији поставља диск у одговарајућу позицију.

Испод табле се налази и мали информациони панел који служи за праћење

статистичких података о агентовим одлукама, уз податке о одабраном агенту и тежини која је одабрана.



Слика 2: Приказ тока игре

Након што се игра заврши, људском играчу се приказују агентове статистике, уз информацију о завршетку партије, као и могућност изласка из апликације или поновног играња партије.

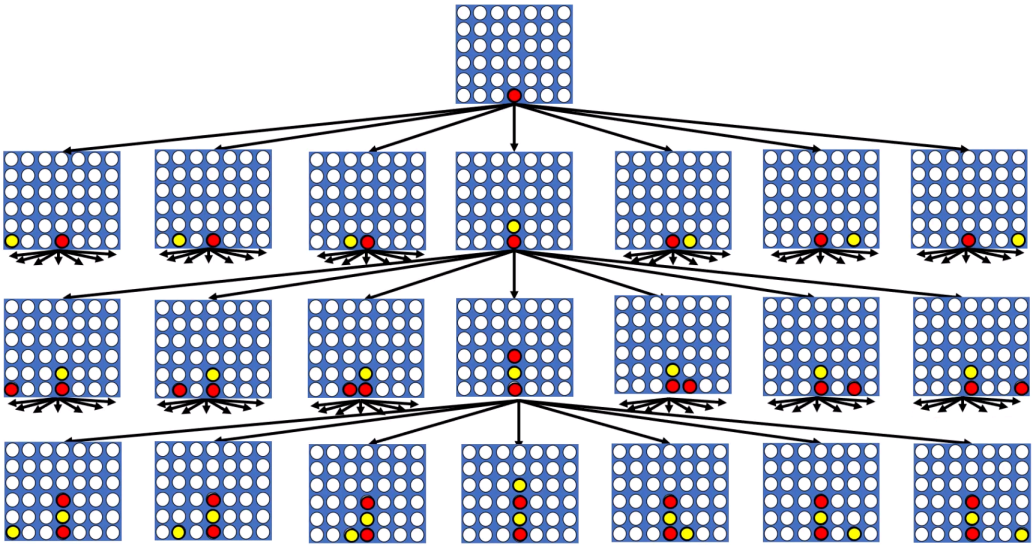
Слика 3: Приказ свих могућих завршетака игре

Опис *Minimax* алгоритма

Minimax алгоритам је рекурзивни алгоритам чији је главни циљ да пронађе оптималан потез тако што максимизује агентову добит, док истовремено претпоставља да ће противник играти рационално и да ће имати за циљ да минимизује сопствени губитак. Алгоритам функционише тако што креира стабло игре у којем се налазе сва могућа стања табле. Сваки чвор у датом стаблу представља стање игре, док свака грана представља један од 6 могућих потеза (одабраних колона) у игри.

С обзиром на сложеност игре и рекурзивну природу алгоритма, дубина стабла игре се може слободно одабрати и она ће представљати све могуће комбинације потеза до тог

одређеног нивоа, па ће и алгоритам само истраживати до одређене, већ задате, дубине.
Самим тим што је циљ самог агента да максимизује сопствену добит и да смањи губитак, рационално је сматрати да ће алгоритам увек бирати путање које му на највећој дубини дају најповољнију евалуацију, чиме је гарантовано да ће алгоритам у датој ситуацији увек бирати оптималан потез. За свако стање које се налази на дну стабла, његове листове, алгоритам одређује колико су та стања повољна за њега и на основу тих вредности доноси одлуку.



Слика 4: Поједностављени приказ стабла игре за *Minimax* алгоритам

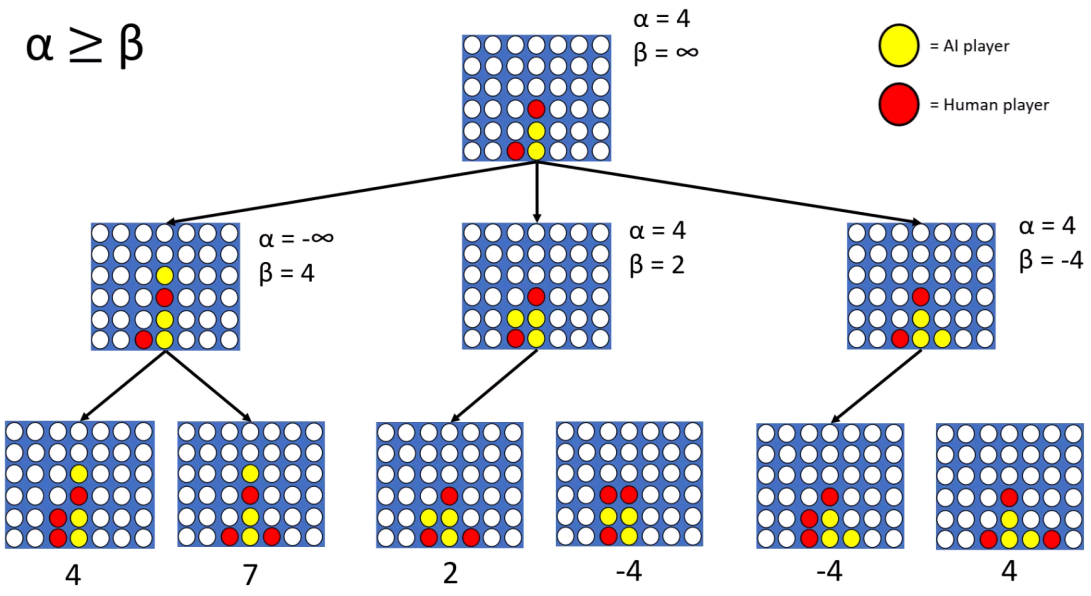
Опис *Alpha-beta pruning* алгоритма

Због постојања великог броја могућности и ограничења компјутерских ресурса, јавила се потреба за оптимизацијом претходног алгоритма.
Alpha-beta pruning представља технику која драстично смањује број чворова које *Minimax* мора да испита, али још увек очувава идентичност самог резултата, само уз значајно скраћивање трајања извршавања. Алгоритам се ослања на два додатна параметра у односу на оригинални *Minimax*:

- *Alpha (α)*: Представља најбољи резултат који максимизирајући играч (агент) може да гарантује на датом нивоу претраге или изнад.
- *Beta (β)*: Представља најбољи резултат који минимизирајући играч (човек) може да гарантује.

Логика одсецања грана је следећа: током претраге, уколико алгоритам наиђе на грану за коју се испостави да је лошија од већ пронађене најбоље опције *(α≥β)*, та грана се одсеца и даље се не истражује.

У неким случајевима се може десити да овај алгоритам не скрати ниједну грану и то се дешава ако су вредности добитака сортиране од најгорег до најбољег за датог играча, због чега алгоритам не може да скрати ниједну грану. Важи и обрнуто, дакле ако су добици сортирани од најбољег до најгорег за датог играча, алгоритам ће скратити максималан број грана чиме се значајно убрзава извршавање.



Слика 5: Поједностављени приказ скраћеног стабла игре за *Alpha-beta pruning* алгоритам

Опис коришћене хеуристике

Хеуристичка функција представља, уз коришћене алгоритме, најбитнији део самог агента, јер му омогућава да процени квалитет тренутног стања табле. У наставку ћемо објаснити како функционише вредновање табле кроз два главна аспекта: стратешке контроле центра и анализу “прозора од четири поља”.

Стратешка контрола центра

Једна од основних стратегија у овој игри јесте управо контрола централних колона, из разлога што оне пружају највећи број могућности за формирање низова у свим правцима (вертикално, хоризонтално или дијагонално), самим тим им се додељује такозване тежине. У нашем случају је ово решено тако што за сваки диск агента у централним колонама додељујемо додатних 10 поена самој вредности стања табле.

```
    # Centar
    center_array = [int(i) for i in list(board[:, COLUMN_COUNT // 2])]
    score += 10 * center_array.count(piece)
    if COLUMN_COUNT % 2 == 0:
        center_array = [int(i) for i in list(board[:, COLUMN_COUNT // 2
+ 1])]
        score += 10 * center_array.count(piece)
```

Прилог 1: Додела тежина централним елементима

Евалуација прозора

Главни део евалуације тренутног стања табле се врши функцијом *evaluate_window* која као параметар прихвата сам прозор који треба проценити и последњи одиграни диск.

Дата функција ради тако што анализира сваки могући прослеђени низ од четири поља (вертикално, хоризонтално и дијагонално) и на основу тога им додељује вредности које су описане у коду испод.

```
def evaluate_window(window, piece):
    score = 0
    opp_piece = AI_PIECE if piece == PLAYER_PIECE else PLAYER_PIECE

    if window.count(piece) == 4:
        score += 100_000
    elif window.count(piece) == 3 and window.count(EMPTY) == 1:
        score += 500
    elif window.count(piece) == 2 and window.count(EMPTY) == 2:
        score += 100

    if window.count(opp_piece) == 4:
        score -= 80_000
    elif window.count(opp_piece) == 3 and window.count(EMPTY) == 1:
        score -= 8000
    elif window.count(opp_piece) == 2 and window.count(EMPTY) == 2:
        score -= 80

    return score
```

Прилог 2: Функција за евалуацију прозора

Поређење времена извршавања

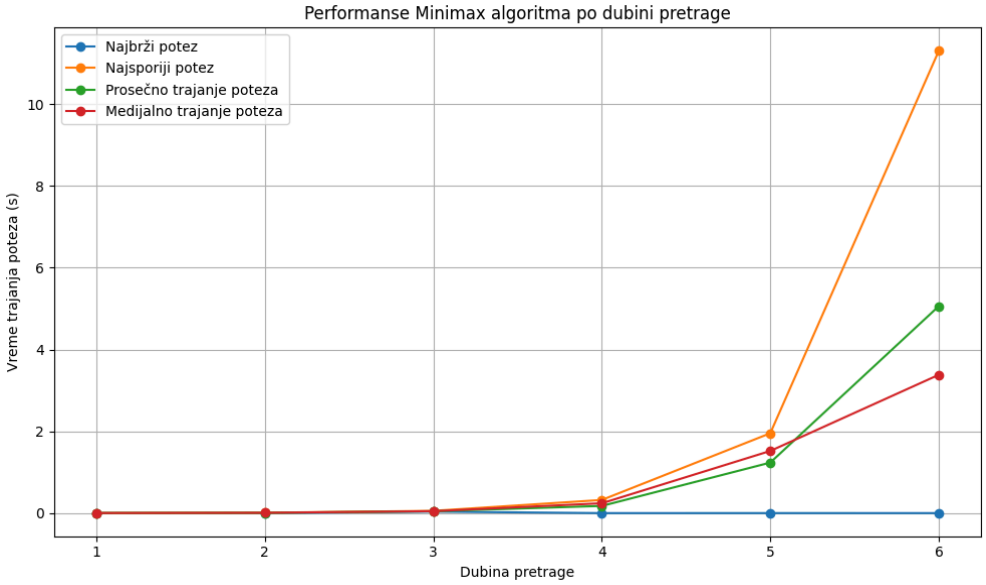
У овом поглављу ћемо изнети податке мерења времена извршавања за оба алгоритма и за њихове различите дубине претраге у стаблу игре, након чега ћемо их упоредити и прокоментарисати. Сва добијена мерења су изражена у секундама и представљају просеке добијене током реалног тестирања партија.
Табеле су форматиране тако да су у колонама наведене дубине претраге у стаблу игре, док су по редовима организоване основне статистичке величине, док саме ћелије представљају њихове вредности.

Minimax алгоритам

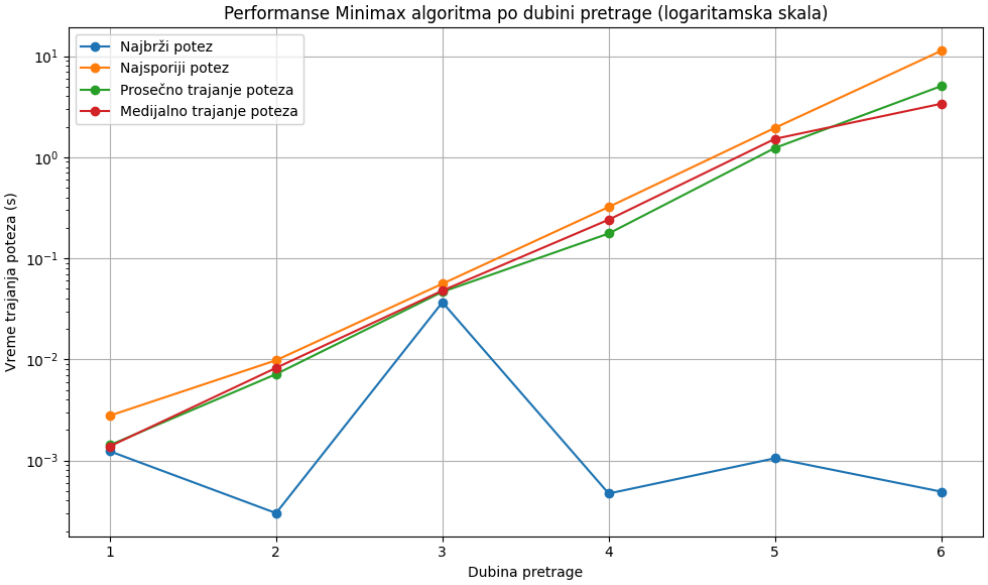
	1	2	3	4	5	6
Просечно време	0.001418	0.007146	0.046582	0.175927	1.235514	5.059492
Најбржи потез	0.001234	0.000301	0.036693	0.000471	0.001052	0.000491
Најспоријe	0.002776	0.009799	0.055981	0.322033	1.95294	11.301826

и потез						
Медијалн о време	0.00138	0.008243	0.047883	0.241001	1.520228	3.381364

Табела 1: Статистика за агента који користи *Minimax* алгоритам



Слика 6: Приказ времена извршавања за *Minimax* алгоритам по дубини претраге



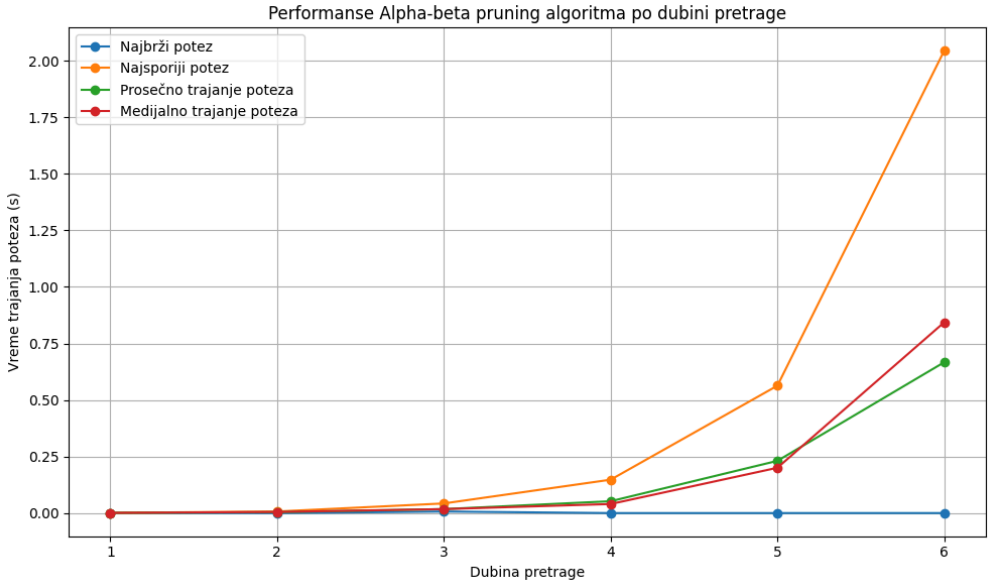
Слика 7: Приказ времена извршавања за *Minimax* алгоритам по дубини претраге - логаритамска скала

Alpha-beta pruning алгоритам

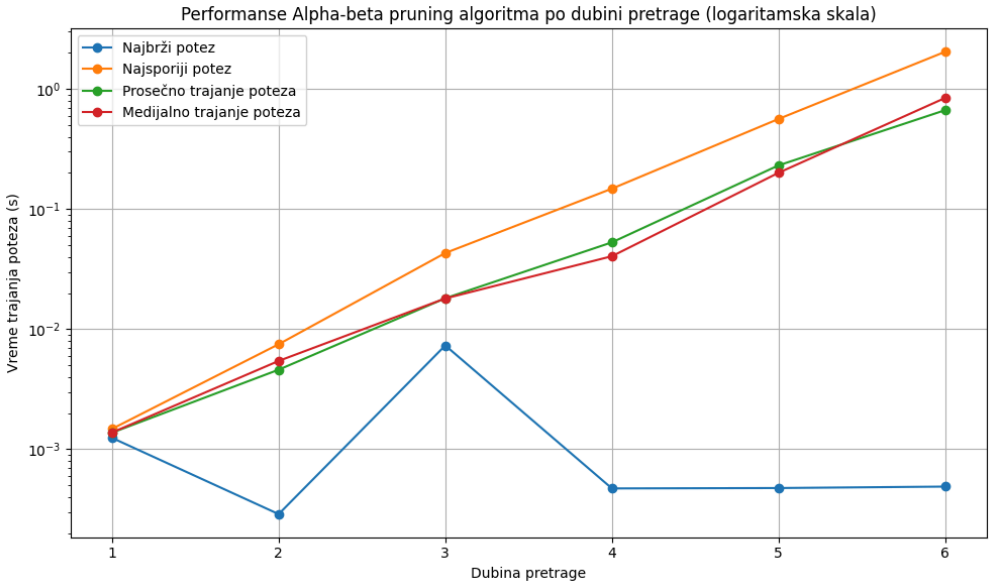
	1	2	3	4	5	6
Просечно време	0.001375	0.004599	0.018104	0.052875	0.231343	0.66708

Најбржи потез	0.001244	0.000288	0.007322	0.000472	0.000476	0.000489
Најспорији потез	0.001479	0.007497	0.043087	0.148059	0.563554	2.043946
Медијално време	0.001379	0.005442	0.018035	0.040528	0.200776	0.843488

Табела 2: Статистика за агента који користи *Alpha-beta pruning* алгоритам

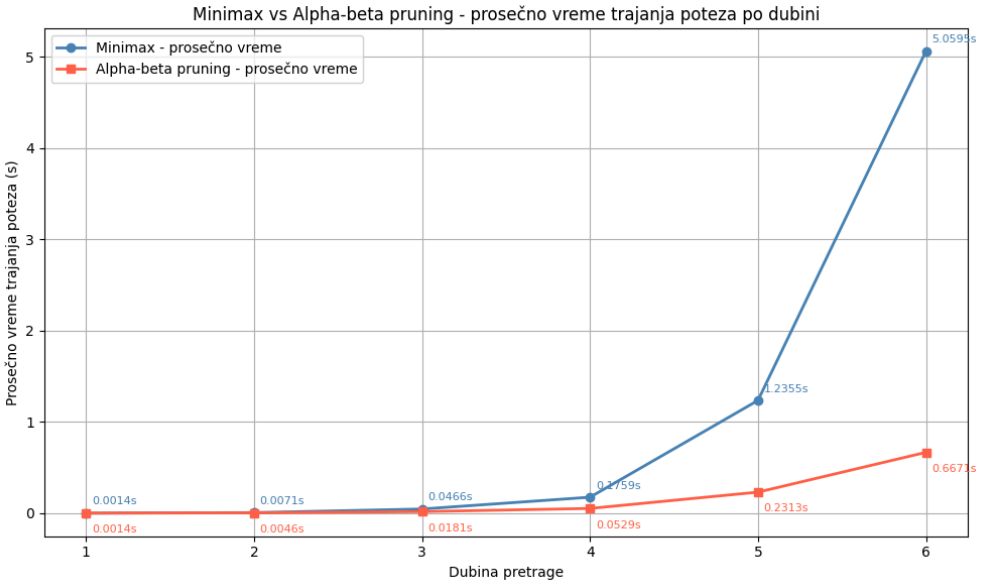


Слика 8: Приказ времена извршавања за *Alpha-beta pruning* алгоритам по дубини претраге

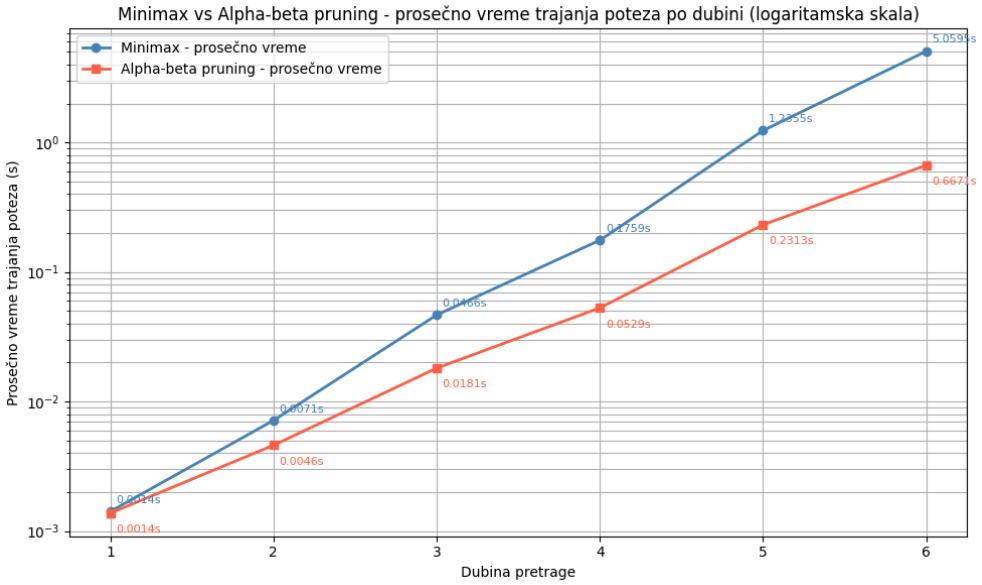


Слика 9: Приказ времена извршавања за *Alpha-beta pruning* алгоритам по дубини претраге - логаритамска скала

Поређење алгоритама



Слика 10: Упоредни приказ просечног времена извршавања за оба алгоритма



Слика 11: Упоредни приказ просечног времена извршавања за оба алгоритма - логаритамска скала

На основу добијених података, могу се извући следећи кључни закључци:

- Ефикасност скраћивања (*Pruning*): Како се дубина повећава, тако *Alpha-Beta pruning* алгоритам постаје све бржи у односу на стандардни *Minimax* алгоритам што указује на то да одбацивањем грана које не утичу на коначну одлуку драстично убрзавамо доношење одлука.
- Експоненцијални раст: Код *Minimax* алгоритма време извршавања драстично расте са повећањем дубине, јер се број чворова које треба испитати експоненцијално увећава, то јест у нашем случају доноси по 6 пута више чворова које треба проверити по сваком додатном нивоу дубине.
- Динамика игре: Први потези агента су увек најспорији јер је табла празна и

број могућих стања је максималан. Како партија одмиче, време доношења одлуке се смањује, јер попуњена поља ограничавају претрагу.

Закључак

Сматрамо да је у оквиру овог пројекта успешно реализовано креирање интелигентног агента за игру *“4 Connect”*, коришћењем неоптимизованог *Minimax* алгоритма, као и његове оптимизоване верзије *Alpha-beta pruning,* чиме смо показали повезаност између области теорије игара и вештачке интелигенције. Кроз процес развоја и тестирања дошли смо до следећих закључака:

- Ефикасност оптимизације: Примена оптимизоване верзије алгоритма се показала као важан фактор за перформансе, чиме је потврђена важност саме оптимизације и у нашем случају је омогућило да агент брже истражује стабло игре до веће дубине у разумном временском року, без негативног утицаја на квалитет потеза.
- Хеуристички значај: Пошто је претрага целокупног стабла игре рачунарски немогућа за веће дубине, хеуристика је била кључна за квалитет самог агента. Увођењем тежина за централна поља, праћењем противникових потеза и проценом сопствених потеза обезбеђена је и дефанзивна и офанзивна игра.
- Динамика времена извршавања: Време доношења одлуке није константно током партије већ је динамично. У почетку партије агент најспорије доноси одлуке због великог броја могућих комбинација, док се временом број комбинација смањује, чиме се брзина доношења одлука убрзава.
- Експоненцијални раст сложености: Пројекат је потврдио да сваким повећањем дубине за 1, број чворова се повећава до 6 пута, што нам указује на хардверске лимитације и неопходност коришћења напредних техника смањења броја грана.